Proof-of-Concept Kit — Microsoft Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 5
- Boundary . . . 5
- Effort . . . 6

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 6
- What Microsoft Defender for Cloud Apps is . . . 6
- What a Proof-of-Concept Kit is . . . 6
- Why this matters . . . 7
- How it works . . . 7
- What you can control . . . 7
- Limitations (be honest with buyers) . . . 7
- Connections and authentication used . . . 8
- The standard deployment model . . . 8

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 8
- 2.1 Defender for Cloud Apps plan available to activate . . . 8
- 2.2 Pilot user group (test users) . . . 9
- 2.3 A discovery source (MDE integration or sample log) . . . 9
- 2.4 A sanctioned app to connect (optional) . . . 9
- 2.5 A test account for benign policy-matching activity . . . 9
- 2.6 Evaluation criteria agreed up front . . . 9
- 2.7 Advanced-hunting read access . . . 10
- Checklist . . . 10

Global Requirements — Defender (category-level) 10
- G1. Microsoft 365 / Entra tenant . . . 10
- G2. Defender licensing . . . 10
- G3. Administrative roles (least privilege) . . . 11
- G4. Onboarding reach to endpoints / identity / workloads . . . 11
- G5. Network access to Defender service endpoints . . . 11
- G6. A pilot ring . . . 11
- G7. Log Analytics workspace (where required) . . . 11
- G8. Response governance . . . 11

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 12
- Naming (set once, reuse verbatim) . . . 12

Step 1 — Assess readiness (read-only) . . . 12
Step 2 — Enable discovery and an audit-mode policy . . . 12
Step 3 — Generate benign test signal . . . 13
Step 4 — Validate and demonstrate the detections . . . 13
Step 5 — Score against your criteria . . . 13
Step 6 — Tear down (clean rollback) . . . 13
Step 7 — Decide: expand or stop . . . 14
5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 14
Test 1 — Audit policy scoped . . . 14
Test 2 — Benign signal generated . . . 14
Test 3 — Cloud Discovery report populated . . . 14
Test 4 — Activity policy matched . . . 14
Test 5 — Activity-log / hunting visibility . . . 15
Test 6 — Alert surfaced . . . 15
Test 7 — Scorecard produced . . . 15
Test 8 — Clean teardown . . . 15
6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 15
Policy scope or action wrong . . . 15
Signal not generated . . . 15
Discovery report empty . . . 16
Policy did not match . . . 16
No activity visibility . . . 16
No alert surfaced . . . 16
Script cannot authenticate . . . 16
Teardown left residue . . . 16
7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 17
Keep the PoC scoped, monitor-first, and reversible . . . 17
Benign signal only — never real exfiltration . . . 17
Use the sample discovery log for a clean demo . . . 17
Keep enforcement off . . . 17
Set the bar before you run . . . 17
Allow warm-up before judging . . . 17
Tear down completely . . . 17
Keep licensing and preview facts current . . . 18
Where to go next . . . 18
8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps 18

Documentation 18
PowerShell scripts (Microsoft Graph Security + KQL) 18
What the buyer receives 19
Outcome 19
Appendix A — PowerShell Scripts 20

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

White-label · Defender

Deciding on Microsoft Defender for Cloud Apps (MDCA) shouldn't rest on a slide deck. This Proof-of-Concept (PoC) Kit stands up a **scoped, safe** proof of concept in your own tenant: it enables MDCA on a small group of **pilot users**, generates **benign, simulated** test signal (Microsoft's own **sample Cloud Discovery log** and benign, policy-matching activity performed by a test account in **audit/monitor** mode — never real data exfiltration or attacks), demonstrates the headline **cloud app security / CASB** capabilities on that signal, scores the result against your evaluation criteria, and then **cleanly tears the PoC back down**.

The kit is written for an administrator with no prior Defender experience. Everything is **pilot-scoped, monitor-first, and reversible**: you scope discovery and policy to a few test users, prove the value, capture a scorecard, and roll it all back — with nothing blocked and no production users affected.

What you get

- A plain-language **Solution Overview** of MDCA / CASB and how a PoC proves it on real-but-safe signal.
- A **Requirements Guide** of the PoC-specific prerequisites (the category-wide ones live in `global-requirements-defender.md`).
- An **Implementation Guide** — exact, ordered steps to enable discovery and an audit-mode policy for pilot users, generate benign signal, demonstrate the detections, score, and tear down.
- Five **PowerShell scripts** (Microsoft Graph Security) with **KQL advanced-hunting** queries: a `-WhatIf`-gated pilot enable, a **benign** test-signal helper, a demo/validate check, a criteria scorecard, and a `-WhatIf`-gated teardown.
- **Testing**, **Troubleshooting**, and **Best Practices** guides.

What this delivers

A concrete, **in-tenant proof** that Microsoft Defender for Cloud Apps delivers CASB — a **Shadow IT / Cloud Discovery** report from a benign sample log, an **activity policy** matching benign test activity in audit mode, an **anomaly/alert** surfaced, the events visible in the activity log and advanced hunting, and a **scorecard** mapping each observed capability to your pass/fail criteria — all rolled back cleanly at the end.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — here scoped as a PoC: assess readiness, enable discovery + an audit-mode policy for pilot users, generate benign signal, validate and score, then either expand or tear down.

Boundary

This is an **evaluation kit**, not a production rollout. It proves MDCA on pilot users in **audit/monitor** mode and then rolls the scope back. Org-wide governance, session controls (Conditional Access App Control), and DLP are separate adjacent solutions (see *Where to go next*). All test signal is **benign and simulated**. Enforcement (blocking/session control) is kept **off** during the PoC; governance decisions remain the customer's security/compliance team's.

Effort

A competent administrator can complete the full PoC — enable, signal, validate, score, and teardown — in a short, focused engagement on a few pilot users. No blocking or session enforcement is applied, and every step is reversible. Some detections and session controls are **plan-gated — verify current availability in your tenant**.

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

What Microsoft Defender for Cloud Apps is

Microsoft Defender for Cloud Apps (MDCA) is Microsoft's **cloud access security broker (CASB)**, managed from the unified Defender portal at `security.microsoft.com`. It gives visibility and control over SaaS usage. Its headline capabilities are:

- **Cloud Discovery (Shadow IT)** — discovers the cloud apps in use from firewall/proxy logs or the Defender for Endpoint integration, and scores their risk.
- **App connectors** — API connections to sanctioned SaaS (Microsoft 365, and third-party apps) for deep activity visibility and governance.
- **Policies** — activity policies, anomaly-detection policies, file/DLP policies, and OAuth-app policies that raise alerts and (optionally) govern.
- **Session controls (Conditional Access App Control)** — real-time, in-session controls (e.g. block download) for sanctioned apps. *Session control requires the appropriate plan and Entra Conditional Access — verify current availability in your tenant.*
- **Activity log & advanced hunting** over `CloudAppEvents` for investigation.

What a Proof-of-Concept Kit is

A PoC Kit is a **customer-runnable evaluation**, not a production rollout and not a sales demo. Its job is to answer one question — *does MDCA actually give us visibility and control over cloud-app usage?* — with evidence, safely, in your own tenant. It does that by:

1. **Enabling** MDCA discovery and an **audit-mode** activity policy scoped to a small group of **pilot users** only.
2. **Generating benign test signal** — Microsoft's **sample Cloud Discovery log** (a documented, safe upload that produces a Shadow IT report) and benign, **policy-matching activity** performed by a test account (e.g. a benign download or an administrative action that the audit policy watches for). **No real data exfiltration, malicious app, or attack is ever used.**

3. **Demonstrating** that the signal produces the expected outcomes — a discovery report, a policy match, an anomaly/alert, and activity-log visibility.

4. **Scoring** what you observed against your evaluation criteria.

5. **Tearing down** — removing the policy and discovery data and rolling the scope back.

Why this matters

Shadow IT and risky SaaS usage are invisible until something goes wrong, and a vendor deck can't show you what MDCA surfaces about *your* tenant's app usage. A scoped, **monitor-first** PoC replaces "trust the deck" with "watch it discover and flag." Because everything runs in **audit mode for pilot users only**, nothing is blocked and no production user is affected — you get proof without imposing controls before you've seen the value.

How it works

1. **Assess** readiness with `POCMDCA-Enable.ps1 -WhatIf` — MDCA plan, admin roles, the discovery source (MDE integration or a sample log), and a defined pilot user group.

2. **Configure & enable** — turn on Cloud Discovery, and create the audit-mode activity policy `MDCA PoC Activity Policy – Pilot Users – Audit` scoped **only** to `MDCA PoC – Pilot Users – Pilot`. Optionally connect one sanctioned app in monitor mode.

3. **Generate benign signal** with `POCMDCA-TestSignal.ps1` — upload the Microsoft **sample discovery log** and perform benign, policy-matching activity as a test account.

4. **Validate & demonstrate** with `POCMDCA-Validate.ps1` — confirm the discovery report populated, the audit policy matched, an alert surfaced, and events appear in `CloudAppEvents`.

5. **Score** with `POCMDCA-Scorecard.ps1` against your criteria.

6. **Tear down** with `POCMDCA-Teardown.ps1 -WhatIf` first, then for real — remove the policy, delete the discovery report/connector, resolve test alerts.

What you can control

• **Pilot users** — exactly which users the policy watches; no production user is affected.

• **Which capabilities to prove** — Cloud Discovery, activity policy, anomaly detection, activity-log visibility.

• **Enforcement posture** — **audit/monitor only** during the PoC; no blocking or session control.

• **Scorecard criteria** — the pass/fail bar you evaluate against.

• **Teardown** — full, reversible removal.

Limitations (be honest with buyers)

• MDCA and several detections/session controls are **license-gated**. *Verify current plan availability in your tenant.*

• **Cloud Discovery** depends on a log source — the MDE integration or an uploaded log (the PoC uses Microsoft's **sample log** for a clean, benign demonstration).

• **Session controls (Conditional Access App Control)** require Entra Conditional Access and the appropriate plan — the PoC keeps enforcement off and proves visibility/policy, noting session control as a verify-and-optional demo.

• Test signal is **benign and simulated only** — a sample log and benign policy-matching activity. This proves the visibility/policy pipeline, not resilience to a real exfiltration.

• This is a **PoC**, not a production rollout — it stays scoped and in audit mode, and is designed to be rolled back. Org-wide governance is a separate solution.

• MDCA improves SaaS visibility and control; it does not replace identity hardening, DLP strategy, or the security team's governance decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Cloud apps**) for discovery, connectors, policies, the activity log, and alerts.

• **Microsoft Graph Security** and the advanced-hunting API (`CloudAppEvents`, `AlertInfo`) for read-only validation and scoring.

• The **Defender for Endpoint → Cloud Apps discovery integration** (optional) or an uploaded Cloud Discovery log as the discovery source.

• Least-privilege roles: **Security Administrator** for configuration, **Security Reader** for read-only work, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

> ***Assess readiness → configure/enable discovery + an audit-mode policy for a pilot user group → generate benign test signal → validate the detections and score → expand or tear down.***
> *Repeatable pattern: **Assess → Configure → Pilot → Validate → Expand**, run here as a safe, reversible, monitor-first PoC.*

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Verify **every** requirement before enabling anything. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running a scoped MDCA proof of concept**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Cloud Apps plan available to activate

Description: A licensed **Defender for Cloud Apps** plan the tenant can activate for the PoC. **Why:** Discovery, policies, and alerts are the capabilities being proven; without the plan MDCA cannot be activated. **Verify:** Admin center → **Billing** → **Licenses** shows Defender for Cloud Apps; `security.microsoft.com` → **Cloud apps** loads. Run `POCMDCA-Enable.ps1 -WhatIf`. *Verify current plan availability in your tenant.* **Implement-if-missing:** Assign the Defender for Cloud Apps plan (often bundled in M365 E5 / E5 Security) before enabling.

2.2 Pilot user group (test users)

Description: A small set of **pilot/test users** grouped as `MDCA PoC – Pilot Users – Pilot`. **Why:** The audit policy watches only this group, so no production user is affected and teardown is clean. **Verify:** An Entra group `MDCA PoC – Pilot Users – Pilot` exists; `POCMDCA-Enable.ps1 -WhatIf` lists membership. **Implement-if-missing:** Create the group and add a handful of test users (including one account you will use to perform benign policy-matching activity).

2.3 A discovery source (MDE integration or sample log)

Description: A **Cloud Discovery** source — the Defender for Endpoint integration, a firewall/proxy log, or Microsoft's **sample discovery log** (used by this PoC for a clean, benign demonstration). **Why:** Cloud Discovery needs a log to produce a Shadow IT report; the sample log makes the PoC self-contained and safe. **Verify:** `security.microsoft.com` → **Cloud apps** → **Cloud discovery**; `POCMDCA-Enable.ps1 -WhatIf` reports the discovery source. **Implement-if-missing:** Enable the MDE-to-MDCA discovery integration, or download Microsoft's sample Cloud Discovery log to upload in Step 3.

2.4 A sanctioned app to connect (optional)

Description: Optionally, one **sanctioned SaaS app** (e.g. Microsoft 365) to connect via an app connector in **monitor** mode. **Why:** A connector gives deep activity visibility so the audit policy can match richer signal; it is optional for a minimal PoC (the sample log alone proves discovery). **Verify:** `security.microsoft.com` → **Cloud apps** → **Connected apps**; `POCMDCA-Enable.ps1 -WhatIf` reports connectors. **Implement-if-missing:** Add one connector in monitor mode (no governance policy) — or skip and rely on the sample discovery log.

2.5 A test account for benign policy-matching activity

Description: A **test account** (in the pilot group) that can perform a benign action the audit policy watches for (e.g. a benign download or administrative activity). **Why:** The activity signal must be generated by a real user session so MDCA observes it; a dedicated test account keeps it attributable and teardown clean. **Verify:** You can sign in as a test account in `MDCA PoC – Pilot Users – Pilot`. **Implement-if-missing:** Provision a test account and confirm it can perform the benign action before Step 3.

2.6 Evaluation criteria agreed up front

Description: A short, written list of what "pass" means — e.g. *the discovery report populated, the audit policy matched the benign activity, an alert surfaced, and the events are queryable.* **Why:** A PoC is only conclusive if the bar is set **before** you run it; the scorecard maps observations to these criteria. **Verify:** A criteria list exists; `POCMDCA-Scorecard.ps1` accepts it as input. **Implement-if-missing:** Draft 5–8

criteria with the evaluating stakeholders before Step 1. *What counts as "good enough" is the customer's decision.*

2.7 Advanced-hunting read access

Description: Permission to run advanced-hunting queries via the portal and the Graph Security API (`ThreatHunting.Read.All`). **Why:** The validate and scorecard scripts confirm the signal landed in `CloudAppEvents` and `AlertInfo`. **Verify:** `security.microsoft.com` → **Hunting** → **Advanced hunting** runs a query; `POCMDCA-Validate.ps1` returns rows. **Implement-if-missing:** Grant the running account **Security Reader** (or higher) and consent to the read-only hunting scope.

Checklist

- Defender for Cloud Apps plan available to activate — `POCMDCA-Enable.ps1 -WhatIf`
- Pilot user group `MDCA PoC – Pilot Users – Pilot` (test users)
- A discovery source (MDE integration or Microsoft sample log)
- (Optional) one sanctioned app connected in monitor mode
- A test account for benign policy-matching activity
- Evaluation criteria agreed and written down — `POCMDCA-Scorecard.ps1`
- Advanced-hunting read access (`ThreatHunting.Read.All`)

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature

needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Follow these steps **in order**. All work is in the **unified Defender portal** (`https://security.microsoft.com` → **Cloud apps**). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model (run as a PoC):*** *Assess readiness, enable discovery + an **audit-mode** policy for **pilot users** only, generate **benign** test signal, validate the detections and score them, then **tear down**. All test signal is Microsoft's sample discovery log and benign policy-matching activity — **never real exfiltration or attacks**. Enforcement (blocking / session control) stays **off**; no production user is affected.*

Naming (set once, reuse verbatim)

- Pilot user group: `MDCA PoC – Pilot Users – Pilot`
- Audit-mode activity policy: `MDCA PoC Activity Policy – Pilot Users – Audit`
- (Optional) app connector: `MDCA PoC Connector – Microsoft365 – Pilot`
- Discovery report: `MDCA PoC Discovery – Pilot – Report`
- Scorecard output file: `MDCA PoC Scorecard – Pilot Users – Validate`
- Test-signal tag (used in notes/hunting): `MDCA-PoC-TESTSIGNAL`

Step 1 — Assess readiness (read-only)

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `POCMDCA-Enable.ps1 -WhatIf` — previews (changes nothing): the MDCA plan, admin roles, whether MDCA is activated, the discovery source, connectors, and whether `MDCA PoC – Pilot Users – Pilot` exists.
3. In **Entra** → **Groups**, create `MDCA PoC – Pilot Users – Pilot` and add a handful of **test** users (including the account you will use to generate benign activity).

Step 2 — Enable discovery and an audit-mode policy

1. `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings** — confirm a discovery source (enable the **Defender for Endpoint integration**, or plan to upload the sample log in Step 3).
2. `security.microsoft.com` → **Cloud apps** → **Policies** → **Policy management** → **Create policy** → **Activity policy**. Name it `MDCA PoC Activity Policy – Pilot Users – Audit`. Scope the filter to `MDCA PoC – Pilot Users – Pilot`, choose a benign, repeatable trigger (e.g. a specific administrative activity or a download threshold), and set **Governance actions = none** (audit only — no alerts-to-block, no session control).
3. Alternatively run `POCMDCA-Enable.ps1` (without `-WhatIf`) — it is `-WhatIf`-gated and guides/marks the audit-policy creation scoped to the pilot group.

4. (Optional) **Cloud apps** → **Connected apps** → **App connectors** → add `MDCA PoC Connector – Microsoft365 – Pilot` in **monitor** mode.

Step 3 — Generate benign test signal

1. Run `POCMDCA-TestSignal.ps1` (or follow its printed steps). It performs only **benign, documented** actions tagged `MDCA-PoC-TESTSIGNAL`:

• **Cloud Discovery:** upload Microsoft's **sample Cloud Discovery log** — `Cloud apps → Cloud discovery → Actions → Create Cloud Discovery snapshot report` → upload the sample log. This produces the Shadow IT report `MDCA PoC Discovery – Pilot – Report` from safe, sample data.

• **Policy-matching activity:** as the test account, perform the **benign** action the audit policy watches for (e.g. download a few non-sensitive files from a monitored app, or perform the benign administrative activity). This is normal activity that the audit policy is configured to notice — nothing is exfiltrated.

1. **Do not** exfiltrate real data, connect a malicious app, or grant a risky OAuth consent. The sample log and benign activity are sufficient.

2. Note the time of each signal for the scorecard latency comparison.

Step 4 — Validate and demonstrate the detections

1. Run `POCMDCA-Validate.ps1` — it confirms (read-only): the discovery report populated, the audit policy `MDCA PoC Activity Policy – Pilot Users – Audit` matched, an alert surfaced (`AlertInfo`), and activity appears in `CloudAppEvents`.

2. In the portal, walk the demo: `Cloud apps → Cloud discovery → Dashboard` shows the Shadow IT report; `Cloud apps → Activity log` shows the benign activity; `Cloud apps → Alerts` (or **Incidents & alerts**) shows the policy match.

3. Confirm **no governance action** was taken (audit only) and no production user affected.

Step 5 — Score against your criteria

1. Run `POCMDCA-Scorecard.ps1 -CriteriaPath .\criteria.json -OutFile "MDCA PoC Scorecard – Pilot Users – Validate.json"`. It maps each observation (discovery populated, policy matched, alert surfaced, activity visible) to your pass/fail criteria and writes the scorecard.

2. Review the scorecard with stakeholders — the decision artefact.

Step 6 — Tear down (clean rollback)

1. Run `POCMDCA-Teardown.ps1 -WhatIf` first to preview, then for real.

2. Teardown: **delete** the policy `MDCA PoC Activity Policy – Pilot Users – Audit`, **delete** the discovery report `MDCA PoC Discovery – Pilot – Report`, **remove** the optional connector, and **resolve** the `MDCA-PoC-TESTSIGNAL` alerts.

3. Optionally delete the pilot group `MDCA PoC – Pilot Users – Pilot`. Keep the scorecard as your record.

Step 7 — Decide: expand or stop

1. If the PoC passed and you choose to proceed, move to an **org-wide MDCA governance / session-control / DLP** engagement — a separate, adjacent solution (see *Where to go next*).
2. If not proceeding, confirm teardown with `POCMDCA-Validate.ps1` (the policy and report should be gone).

Order of operations

`Assess (-WhatIf) → enable discovery + audit-mode policy (pilot users) → upload sample discovery log + benign policy-matching activity → validate + demonstrate → score vs criteria → teardown (-WhatIf then real) → decide expand or stop`

5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Run these checks during the PoC (Steps 2–6 of the Implementation Guide). Each test has a **pass/fail gate**. All signal is **benign** (Microsoft sample discovery log + benign policy-matching activity), and everything runs in **audit mode**. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Audit policy scoped

Procedure: Run `POCMDCA-Enable.ps1 -WhatIf`, then confirm `MDCA PoC Activity Policy – Pilot Users – Audit` exists, is scoped to the pilot group, and has **no governance action** (audit only). **Pass:** The policy targets the pilot users only and takes no action. **Fail:** Scope/action wrong — see Troubleshooting *Policy scope or action wrong*.

Test 2 — Benign signal generated

Procedure: Run `POCMDCA-TestSignal.ps1`; confirm the sample discovery log uploaded and the benign policy-matching activity was performed. **Pass:** Both benign signals were generated; no real data exfiltration used. **Fail:** A signal did not generate — see Troubleshooting *Signal not generated*.

Test 3 — Cloud Discovery report populated

Procedure: `Cloud apps → Cloud discovery → Dashboard` shows `MDCA PoC Discovery – Pilot – Report`; `POCMDCA-Validate.ps1` confirms discovery data. **Pass:** The Shadow IT report populated from the sample log with app risk scores. **Fail:** Empty report — see Troubleshooting *Discovery report empty*.

Test 4 — Activity policy matched

Procedure: Confirm the audit policy matched the benign activity (`Cloud apps → Alerts` or the policy's matches); `POCMDCA-Validate.ps1` checks `AlertInfo`. **Pass:** The policy matched and recorded the

benign activity in audit mode. **Fail:** No match — see Troubleshooting *Policy did not match*.

Test 5 — Activity-log / hunting visibility

Procedure: `Cloud apps → Activity log` shows the benign activity; `POCMDCA-Validate.ps1` runs KQL over `CloudAppEvents`. **Pass:** The activity is visible in the log and hunting returns recent rows. **Fail:** Nothing visible — allow warm-up; see Troubleshooting *No activity visibility*.

Test 6 — Alert surfaced

Procedure: Confirm an alert/anomaly surfaced for the policy match in **Incidents & alerts** / **Cloud apps** → **Alerts**. **Pass:** An alert is raised (audit mode still alerts; it just does not govern). **Fail:** No alert — see Troubleshooting *No alert surfaced*.

Test 7 — Scorecard produced

Procedure: Run `POCMDCA-Scorecard.ps1` and confirm `MDCA PoC Scorecard – Pilot Users – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** Missing observations — re-run the relevant validate step first.

Test 8 — Clean teardown

Procedure: Run `POCMDCA-Teardown.ps1 -WhatIf`, then for real; re-run `POCMDCA-Validate.ps1`. **Pass:** The policy, discovery report, and connector are removed and test alerts resolved. **Fail:** Residue remains — see Troubleshooting *Teardown left residue*.

Exit gate

Tests 1–7 pass to call the PoC conclusive; Test 8 passes to close it out cleanly. Keep the scorecard file as the decision record.

6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Common field issues, in **symptom** → **cause** → **fix** form. Run `POCMDCA-Enable.ps1 -WhatIf` and `POCMDCA-Validate.ps1` first — most causes surface there. All test signal is benign (sample discovery log + benign policy-matching activity) and everything runs in audit mode.

Policy scope or action wrong

Symptom: The activity policy applies to more than the pilot users, or it has a governance action. **Cause:** The policy filter was not scoped to `MDCA PoC – Pilot Users – Pilot`, or a governance action was left set. **Fix:** Edit `MDCA PoC Activity Policy – Pilot Users – Audit` — restrict the filter to the pilot group and set **Governance actions = none**. Re-run `POCMDCA-Enable.ps1 -WhatIf`.

Signal not generated

Symptom: The sample log did not upload, or the benign activity did not register. **Cause:** The upload used the wrong format/source, or the test account is not in the pilot group. **Fix:** Use Microsoft's sample Cloud Discovery log with a matching data source in the snapshot report; confirm the test account is in `MDCA PoC – Pilot Users – Pilot`; re-run `POCMDCA-TestSignal.ps1`.

Discovery report empty

Symptom: Test 3 fails — `MDCA PoC Discovery – Pilot – Report` shows no data. **Cause:** The snapshot report is still processing, or the log/source mismatched. **Fix:** Allow processing time (discovery reports take time to parse); confirm the sample log matched the chosen data source; re-check `Cloud apps → Cloud discovery → Dashboard`.

Policy did not match

Symptom: Test 4 fails — the audit policy did not match the benign activity. **Cause:** The activity did not meet the policy filter/threshold, the connector is not yet returning activity, or signal has not propagated. **Fix:** Confirm the benign action matches the policy's trigger (threshold/activity type); if using a connector, confirm it is returning activity; allow warm-up and re-run `POCMDCA-Validate.ps1`.

No activity visibility

Symptom: Test 5 fails — `Cloud apps → Activity log` or `CloudAppEvents` shows nothing. **Cause:** No connector/discovery source is feeding activity yet, or hunting tables have not populated. **Fix:** Confirm a discovery source or connector is active; allow warm-up (tables populate over minutes to hours); confirm `ThreatHunting.Read.All` consent; re-run the validate script.

No alert surfaced

Symptom: Test 6 fails — the policy matched but no alert appears. **Cause:** The policy's alerting is off, or the alert is still propagating to Incidents & alerts. **Fix:** Confirm the policy raises an alert on match (alerting can be independent of governance actions); allow propagation time; check both `Cloud apps → Alerts` and **Incidents & alerts**.

Script cannot authenticate

Symptom: `POCMDCA-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, read scopes were not consented, or the account lacks the role. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm **Security Reader** (or Security Administrator for enable/teardown). Re-run.

Teardown left residue

Symptom: After `POCMDCA-Teardown.ps1`, the policy, report, or connector remains. **Cause:** Teardown ran in `-WhatIf` only, or an item must be removed in the portal. **Fix:** Run teardown **without** `-WhatIf`; delete the policy, the discovery report `MDCA PoC Discovery – Pilot – Report`, and the connector in `Cloud apps`; resolve the `MDCA-PoC-TESTSIGNAL` alerts; re-confirm with `POCMDCA-Validate.ps1`.

7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Field-proven guidance for running a clean, safe, conclusive MDCA proof of concept.

Keep the PoC scoped, monitor-first, and reversible

- Scope the activity policy to **pilot users only** (`MDCA PoC – Pilot Users – Pilot`) and keep it in **audit mode with no governance action**. The value is that you prove visibility without blocking anyone.
- Preview every change with `-WhatIf` first (`POCMDCA-Enable.ps1`, `POCMDCA-Teardown.ps1`).

Benign signal only — never real exfiltration

- Use Microsoft's **sample Cloud Discovery log** and **benign policy-matching activity** exclusively. They exercise discovery and policy without moving real data.
- **Never** exfiltrate real data, connect a malicious app, or grant a risky OAuth consent to "test." A PoC proves the product surfaces the signal; it is not a data-loss exercise.

Use the sample discovery log for a clean demo

- The **sample log** makes the PoC self-contained: it produces a realistic Shadow IT report without needing a real firewall/proxy feed, so you can demonstrate discovery immediately and tear it down cleanly.

Keep enforcement off

- Do **not** turn on blocking or **session controls (Conditional Access App Control)** during the PoC. Session control changes the user experience in real time and requires Conditional Access wiring — prove visibility and policy first; enforcement is a production decision for the security/compliance team.

Set the bar before you run

- Agree the **evaluation criteria** in writing first (Requirement 2.6). Feed them into `POCMDCA-Scorecard.ps1` so the verdict is objective.

Allow warm-up before judging

- The discovery report, activity log, `CloudAppEvents`, and anomaly detections populate over **minutes to hours**. Give the pipeline time before scoring a "fail," and use `POCMDCA-Validate.ps1` to check when signal has arrived.

Tear down completely

- Delete the policy, the discovery report, and the optional connector; resolve the `MDCA-PoC-TESTSIGNAL` alerts. Then confirm with `POCMDCA-Validate.ps1`. Keep only the scorecard.

Keep licensing and preview facts current

- MDCA detections and session controls are license-gated, and availability changes. **Verify current plan availability in your tenant** before assuming a capability — and if session control is unlicensed, record it as "not licensed," not "failed."

Where to go next

Once the PoC proves MDCA, natural next steps in the catalogue:

- The **Cloud App DLP / session-control governance** solution to move from audit to real governance org-wide.
- The **Defender XDR Deployment & Unified Onboarding** solution to correlate MDCA with Endpoint, Identity, and Office.
- The **Entra ID Conditional Access** solution to wire up Conditional Access App Control for session controls.
- The **Sentinel SOC/XDR integration** solution to stream MDCA into a SIEM for long-term hunting.

8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud Apps

Documentation

Document	Purpose
Executive Summary	What the PoC proves, the boundary, and the effort
1. Solution Overview	What MDCA / CASB is and how a scoped, safe, monitor-first PoC proves it
2. Requirements Guide	PoC-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable discovery + audit policy, generate benign signal, validate, score, tear down
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Safe-PoC guidance and where to go next

PowerShell scripts (Microsoft Graph Security + KQL)

Script	What it does
`POCMDCA-Enable.ps1`	`-WhatIf`-gated pilot enable — previews readiness, then enables discovery and an audit-mode activity policy scoped to `MDCA PoC – Pilot Users – Pilot` (safe preview by default)
`POCMDCA-TestSignal.ps1`	**Benign** test-signal helper — uploads Microsoft's sample Cloud Discovery log and guides benign policy-matching activity (no real exfiltration, ever)
`POCMDCA-Validate.ps1`	Read-only demo/validate — confirms the discovery report, policy match, alert, and activity-log / hunting visibility
`POCMDCA-Scorecard.ps1`	Scores observations against your evaluation criteria and writes `MDCA PoC Scorecard – Pilot Users – Validate`
`POCMDCA-Teardown.ps1`	`-WhatIf`-gated clean rollback — deletes the policy, discovery report, and connector, resolves test alerts

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — run here as a safe, reversible, monitor-first PoC.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A concrete, in-tenant proof that Microsoft Defender for Cloud Apps delivers CASB — a Shadow IT discovery report from a benign sample log, an audit-mode activity policy matching benign test activity, an alert surfaced, and full activity-log visibility — scored against your criteria and cleanly rolled back.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

POCMDCA-Enable.ps1

Purpose: Prepares the MDCA proof of concept for PILOT USERS only, in AUDIT/MONITOR mode. It previews readiness (MDCA plan, admin roles, activation state, discovery source, connectors, pilot group) and guides the audit-mode activity policy creation scoped to the pilot group. MDCA policies are authored in the portal; this script marks intent behind -WhatIf and never turns on blocking or session control. HOW TO RUN Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDCA-Enable.ps1 -WhatIf Confirm the guided enable: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Enable.ps1 -PilotGroup "MDCA PoC - Pilot Users - Pilot" Requires Microsoft.Graph.Authentication and Security Administrator. EXPECTED RESULTS A readiness summary and guided, audit-only policy steps scoped to the pilot users. No enforcement is enabled. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped, audit-mode only. Reverse with POCMDCA-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDCA PoC - Pilot Users - Pilot', [string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit')

How to run: Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDCA-Enable.ps1 -WhatIf Confirm the guided enable: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Enable.ps1 -PilotGroup "MDCA PoC - Pilot Users - Pilot" Requires Microsoft.Graph.Authentication and Security Administrator. EXPECTED RESULTS A readiness summary and guided, audit-only policy steps scoped to the pilot users. No enforcement is enabled. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped, audit-mode only. Reverse with POCMDCA-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PilotGroup = 'MDCA PoC - Pilot Users - Pilot', [string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit')

POCMDCA-Scorecard.ps1

Purpose: Turns the PoC observations into an objective, shareable scorecard. It re-checks the key MDCA capabilities (cloud-app activity visible, audit-policy alert surfaced) via read-only advanced hunting, maps each to the pass/fail criteria you agreed up front, and writes a JSON + console scorecard. Confirm the Cloud Discovery report in the portal for the discovery criterion (report state is not in the hunting tables). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDCA-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDCA PoC Scorecard - Pilot Users - Validate.json" Sample criteria.json: [{ "id":"activity", "label":"Cloud-app activity visible", "capability":"activity", "required":true }, { "id":"alert", "label":"Audit policy alert surfaced", "capability":"alert", "required":true }, { "id":"discovery","label":"Shadow IT report populated", "capability":"portal", "required":true }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDCA PoC Scorecard - Pilot Users - Validate.json', [int]$LookbackHours = 12)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDCA PoC Scorecard - Pilot Users - Validate.json" Sample criteria.json: [{ "id":"activity", "label":"Cloud-app activity visible", "capability":"activity", "required":true }, { "id":"alert", "label":"Audit policy alert surfaced", "capability":"alert", "required":true }, { "id":"discovery","label":"Shadow

IT report populated", "capability":"portal", "required":true }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDCA PoC Scorecard - Pilot Users - Validate.json', [int]$LookbackHours = 12)

POCMDCA-Teardown.ps1

Purpose: Returns the tenant to its pre-PoC state. It guides removal of: - the audit-mode activity policy 'MDCA PoC Activity Policy - Pilot Users - Audit', - the Cloud Discovery snapshot report 'MDCA PoC Discovery - Pilot - Report', - the optional app connector, and reminds you to resolve the MDCA-PoC-TESTSIGNAL alerts. MDCA policy/report objects are removed in the portal; this script marks intent behind -WhatIf. Nothing was ever enforced, so there is no blocking to undo. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Teardown.ps1 -WhatIf Apply (prints the guided steps and marks intent): powershell -ExecutionPolicy Bypass -File .\POCMDCA-Teardown.ps1 EXPECTED RESULTS The audit policy, discovery report, and connector removed, and test alerts resolved. .NOTES White-label. Reversible teardown. Audit-mode only — no enforcement to roll back. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit', [string]$ReportName = 'MDCA PoC Discovery - Pilot - Report', [string]$ConnectorName = 'MDCA PoC Connector - Microsoft365 - Pilot', [string]$Tag = 'MDCA-PoC-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Teardown.ps1 -WhatIf Apply (prints the guided steps and marks intent): powershell -ExecutionPolicy Bypass -File .\POCMDCA-Teardown.ps1 EXPECTED RESULTS The audit policy, discovery report, and connector removed, and test alerts resolved. .NOTES White-label. Reversible teardown. Audit-mode only — no enforcement to roll back. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit', [string]$ReportName = 'MDCA PoC Discovery - Pilot - Report', [string]$ConnectorName = 'MDCA PoC Connector - Microsoft365 - Pilot', [string]$Tag = 'MDCA-PoC-TESTSIGNAL')

POCMDCA-TestSignal.ps1

Purpose: Produces harmless cloud-app signal so the MDCA pipeline has something to surface. It: 1. Guides upload of Microsoft's SAMPLE Cloud Discovery log to produce a Shadow IT report from safe, sample data (Cloud apps -> Cloud discovery -> Create snapshot report). 2. Guides a BENIGN, policy-matching action performed by the test account (e.g. a small, non-sensitive download or a benign administrative activity the audit policy watches for). NO real data exfiltration, malicious app connection, or risky OAuth grant is ever performed. -WhatIf shows what it would do. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDCA-TestSignal.ps1 -WhatIf Print the benign signal steps: powershell -ExecutionPolicy Bypass -File .\POCMDCA-TestSignal.ps1 EXPECTED RESULTS A discovery report populates and the audit policy matches the benign activity, for the PoC to validate. .NOTES White-label. BENIGN signal only (sample discovery log + benign policy-matching activity). Never real exfiltration. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$Tag = 'MDCA-PoC-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDCA-TestSignal.ps1 -WhatIf Print the benign signal steps: powershell -ExecutionPolicy Bypass -File .\POCMDCA-TestSignal.ps1 EXPECTED RESULTS A discovery report populates and the audit policy matches the benign activity, for

the PoC to validate. .NOTES White-label. BENIGN signal only (sample discovery log + benign policy-matching activity). Never real exfiltration. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$Tag = 'MDCA-PoC-TESTSIGNAL')

POCMDCA-Validate.ps1

Purpose: Read-only validation and live-demo helper for the MDCA proof of concept. Using the Microsoft 365 Defender advanced-hunting API (via Microsoft Graph Security), it confirms the benign test signal surfaced as: - CloudAppEvents — the benign activity is visible in the activity/hunting data - AlertInfo — the audit policy raised an alert (audit mode still alerts) It also points to the Cloud Discovery dashboard and activity log for the visual demo. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDCA-Validate.ps1 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability, plus portal paths for the discovery / activity-log demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([int]$LookbackHours = 6)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDCA-Validate.ps1 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability, plus portal paths for the discovery / activity-log demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([int]$LookbackHours = 6)